

Enumeracion para privesc en Linux

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Enumerar para privesc es recorrer de forma sistemática todos los vectores posibles en un orden que va de lo más rentable a lo más laborioso. Herramientas como linpeas lo automatizan y colorean los hallazgos, pero entender que representa cada resultado es lo que convierte 'output' en 'vector'. Es el paso previo obligado del privesc-modelo.

Adivinar pierde tiempo. Un recorrido fijo (sudo, SUID, capabilities, cron, servicios, credenciales, kernel) garantiza que no te saltas nada. linpeas resalta en rojo/amarillo lo probable, pero da falsos positivos: hay que validar cada pista contra el modelo mental.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Justo tras conseguir shell y estabilizarla (tty-stabilization). Corre primero los checks rápidos manuales (sudo -l, id) y luego lanza linpeas para lo exhaustivo.

Cómo identificarla

- linpeas marca en rojo/amarillo (99% PE vectors) un binario, servicio o fichero.
- pspy muestra procesos o cron que arrancan como root de forma periódica.
- Un hallazgo que encaja con el modelo: algo root que puedes leer/escribir/influir.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Corre sudo -l e id primero (gratis); después linpeas para el resto.

Evidencia esperada: Al menos un hallazgo resaltado en rojo/amarillo en linpeas, o una entrada NOPASSWD en sudo -l.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
./linpeas.sh -a 2>/dev/null | tee linpeas.txt
```

Objetivo y contexto: Automatiza cientos de checks de privesc en un pase. -a es el modo agresivo; guardar la salida (tee) te deja releerla sin volver a correrlo.

Resultado esperado: Fijate en lo marcado en rojo/amarillo, sobre todo 'sudo', 'SUID', 'Capabilities', 'Cron', 'Interesting files'. Valida cada uno; hay falsos positivos.

Capa 2: prueba documentada

```
./pspy64 -pf -i 1000
```

Objetivo y contexto: Muestra procesos y comandos que arrancan sin necesitar root, ideal para ver cron y tareas que ps puntual no capta. Revela que ejecuta root de forma periodica.

Resultado esperado: Comandos que aparecen cada X segundos con uid=0. Si alguno llama a un script o binario que puedes escribir, tienes un vector tipo cron-abuse.

Anatomía de los payloads

La primera prueba es `./linpeas.sh -a 2>/dev/null | tee linpeas.txt`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Al menos un hallazgo resaltado en rojo/amarillo en linpeas, o una entrada NOPASSWD en sudo -l.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Checks rapidos manuales: id, sudo -l, SUID (suid), capabilities (capabilities).
- Lanza linpeas y lee de arriba a abajo, priorizando lo resaltado.
- Usa pspy para cazar tareas periodicas y procesos que no ves con ps puntual.
- Cada pista -> validala contra un vector concreto (cron, PATH, servicio) antes de explotar.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Ya recorriste todos los vectores manualmente y linpeas no aporta nada nuevo.

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Antes de explotar hay que enumerar en orden fijo; un script automatiza, pero saber que mira cada check es lo que te hace resolver. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (enum-privesc-linux)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.